

単一の演算子から導出されるすべての基本関数

アンドジェイ・オドリウォレク理論物理学研究所、ヤゲウォ大学、ポーランド・クラ
クフ 30-348E-mail: andrzej.odrzywolek@uj.edu.pl 2026年4月7日

要旨

デジタルハードウェアにおけるブール論理のすべての演算は、単一の2入力ゲートで実現可能である。これに相当する基本演算は連続数学の分野では知られておらず、 $\sin$ 、 $\cos$ 、 V 、 $\log$ といった基本関数の計算には常に複数の異なる演算が必要であった。本研究では、単一の二項演算子によって、

$$\text{eml}(x, y) = \exp(x) - \ln(y),$$

定数1と共に、科学計算機の標準的な演算レパートリーを生成できることを明らかにする。これには、 e 、 π 、 i といった定数、 $+$ 、 $-$ 、 $\times$ 、 $/$ などの算術演算、および指数演算、さらには通常の超越関数や代数関数が含まれる。例えば、 $e^x = \text{eml}(x, 1)$
 $\ln x = \text{eml}(1, \text{eml}(\text{eml}(1, x), 1))$ といった関係が成立し、他のすべての演算についても同様である。このような演算子が存在しうことは予想されておらず、私は系統的な網羅的探索によってこれを発見し、科学計算機の具体的な基礎演算として十分であることを構成的に証明した。EML (Exp-Minus-Log) 形式では、あらゆるこのような式が同一のノードからなる二分木として表現され、文法は $S \rightarrow 1 | \text{eml}(S, S)$ のように極めて単純になる。この統一的な構造により、勾配ベースの記号回帰も可能となる。EML木を訓練可能な回路として扱い、標準的な最適化アルゴリズム (Adam) を用いることで、浅い木の深さ (最大4層) までの数値データから基本関数の閉形式表現を正確に復元できることを示す。同じアーキテクチャは任意のデータに適合可能であるが、生成法則が基本的なものである場合、正確な数式を復元できる可能性がある。

要約段落

指数関数、対数関数、三角関数などの基本関数は、STEM教育における標準的な語彙である。それぞれに独自の規則があり、科学計算機には専用のボタンが用意されている。我々の導出では、これらの関数を同時に多数用いるが、これらは互いに冗長性が高く、例えば $\sin x = \cos(x - \pi/2)$ $\sqrt{x} = x^{1/2}$ のように相互に表現可能であることが知られている。これらは定量科学の主力であり、基礎法則や経験則に現れるほか、微分方程式解法、数値積分、フーリエ解析などの数値計算手法の中核をなす[1]。デジタル電子回路においては、驚くべき普遍性の事実が存在する：単一の2入力ゲートであるNAND（シェッファーストロック）だけで、あらゆるブール回路を構築可能である[2]。連続数学にはこのような基本要素が存在せず、計算機には多くの異なるボタンが必要となる。古典的な削減手法——対数表[3, 4]やスライドルールから、オイラーの公式[5]を経てexp-log表現[6]と代数的付加[7]に至るまで——はいくつかの基本関数に還元できるが、それ以上の削減は行われていない。それにもかかわらず、この見かけ上の多様性が本質的なものなのか、それともより単純な生成基礎が存在するのかは依然として不明である。本研究では、演算子 $\text{eml}(x, y) = \exp(x) - \ln(y)$ と定数1を組み合わせることで、まさにこの目的を達成できることを示す：これは算術演算、標準的なすべての基本超越関数、および整数、分数、根号、 e 、 π 、 i などの定数を再構築するものである。より簡潔に言えば、2つのボタン（1とeml）を備えた計算機で、通常の科学計算機が実行できるあらゆる計算が可能となる。EML演算子の存在は、基本関数が従来認識されていたよりもはるかに単純なクラスに属することを明らかにしている。すべてのEML式は同一のノードからなる二分木として表現され、極めて単純な文法が得られる： $S \rightarrow 1 \mid \text{eml}(S, S)$ これは文脈自由言語であり、完全二分木やカタラン構造といったよく研究された組合せ論的对象と同型である。基本式は同一要素から構成される回路[8]となり、これはNANDゲートから構築されるデジタルハードウェアと同様である。この統一的な表現方法により、連続記号回帰[9, 10]のための完全かつ規則的な探索空間が提供される：パラメータ化されたEML木は標準的な勾配法によって最適化可能であり、生成法則が基本的なものである場合、学習された重みは正確な閉形式表現に収束する。要するに、単一の訓練可能なアーキテクチャが、データから任意の基本式を[11] 発見する可能性を秘めているのである。EML演算子は氷山の一角に過ぎないかもしれない。予備的な探索では、すでにさらに強力な特性を持つ関連演算子が発見されており、これには区別すべき定数を必要としない三元演算子も含まれる。

意義の表明

誰もが学校で多く数学的操作を学ぶ：分数、根号、対数関数、そして三角関数 $(+, -, \times, /, \sqrt{}, \sin, \cos, \log, \dots)$ はそれぞれ独自の規則を持ち、科学計算機には専用のボタンが用意されている。高等数学の研究により、これらの多くが互いに冗長であることが明らかになっている：例えば三角関数は複素指数関数に還元可能である。この還元はどこまで可能だろうか？我々は、この還元が究極まで可能であること、すなわち単一の演算子 $\text{eml}(x, y)$ がこれらすべてを置き換えられることを示す。EML演算子と数字1の2つのボタンを備えた計算機で、通常科学計算機が実行できるあらゆる計算が可能となる。これは単なる数学的トリックではない。単一の反復可能な要素で十分であるため、数学的表現は均一な回路となり、同一のトランジスタから構築される電子回路と同様に、科学計算における数式の符号化、評価、発見に新たな道を開くものである。

1 序論

単一で再利用可能な基本演算子は、数学、工学、さらには生物学においても、美的観点と実用面の両方で不釣り合いに大きな役割を果たしている。広く知られている古典的な例として、ブール0/1論理におけるNANDゲート（およびその双対であるPeirceアロー、すなわち論理NOR）[2, 12]、正および負のフィードバックプロセスにおけるオペアンプ[13]、そして近年では深層学習における整流線形ユニット（ReLU）「ランプ」活性化関数[14] [15]などが挙げられる。また、Wolframの単一公理[16]、組み合わせ論理におけるK,Sコンビネータ[17, 18]、インタラクションコンビネータ[19]、シェファーストロークのファジィ版[20]なども言及に値する。その他のよく知られた例として、単一命令セットコンピュータ（OISC）がある。例えばSUBLEQ[21]、ConwayのFRACTRAN[22]、そしてルール110セルオートマトン[16, 23]などがその例である。

単一の十分な演算子あるいは要素が存在することが概念的に重要であるかどうかについては議論がある[24]。しかしながら、理解、教育、そして一般向けの説明におけるその価値は計り知れない。実際、古典的な一階述語論理はNANDを特別に取り上げる必要はない。XOR、AND、NOTなどの冗長な接続詞の完全なファミリーを用いても、まったく問題なく機能するのである。これらの接続詞は単に応用上の利便性のために導入されているに過ぎない。しかし、これらのすべてが単一の基本演算子から定義可能であるという認識は、衝撃的で概念的に深い構造的事実である。同様の認識の変化は、進化生物学においてDNAとRNA[25]がほぼ普遍的な情報担体として認識されるようになった過程にも見られる。

シェッファー型の要素は稀であり、その発見には通常、時間、計算資源、労力、そして多少の運が必要となる。これらの要素は一見無関係な分野、例えば離散幾何学などにも現れる。最近発見された「einstein1問題」の解決は、タイル張り研究コミュニティに大きな影響を与えた[26]。筆者はすでにこのような要素の一つを保有しており、それは(2,3,5)球面メビウス三角形から導出されたイグルー構造用ブロックである[27]。

多くの学生にとって馴染み深い基本的な関数¹は、しばしば忌避される正弦関数と余弦関数を象徴しているが、これらは数量的推論において中心的な役割を果たす。これらの関数は、整数、有理数、 π や e などの数学定数と、四つの基本算術演算(+, −, ×, /)および指数演算を用いて、数え切れないほどの組み合わせが可能である。基本的な関数はSTEM教育の核心であり、現代の科学技術文明の基盤を形成している。これには単純な数式や経験則モデルだけでなく、ほとんどの数値アルゴリズムも含まれる。例えば、常微分方程式ソルバーRK45[28]、積分規則（ガウス積分、タンハ・シン積分など）[1, 29]、そして高速フーリエ変換[30]などが挙げられる。これらのアルゴリズムで使用されるトゥイドル因子も基本的な関数である。現代のほぼすべてのプログラミング言語では、基本的な関数を用いた有限式がサポートされている²。標準的な関数や算術演算の多くが重複している（例えば $\tan x = \sin x / \cos x$ $\sqrt{x} = x^{1/2}$ $\arcsin x = \ln(x + \sqrt{1 + x^2})$ $x/y = x \times y^{-1}$ など）こと、特に複素数領域において（例えば $\cosh x = \cos(ix)$ $\sinh ix = i \sin x$ など）、このことは広く知られているが、これまで単一の基本演算子の候補として考慮されたことはなかった。この観点から基本的な算術演算を理解する上での歴史的に重要な二つのマイルストーンは、(i)対数の発見とそれに続く表の作成[3, 4]、そしてスライドルールの開発、(ii)オイラーの公式 $e^{i\pi} + 1 = 0$ [5]である。対数は乗算を加算に変換するために導入された。exp-log

1 ドイツ語の「ein Stein」（「一枚のタイル」の意）であり、アルバート・アインシュタインに由来するものではない。

2 指数演算は、四つの基本算術演算とは異なり、ほとんどの場合プリミティブ演算子として組み込まれているのに対し、専用の中置演算子としてではなく、ライブラリ関数を通じて提供される高次演算として扱われることが多かった。

このペアにより、相互に表現することが可能となる:

$$x \times y = e^{\ln x + \ln y}, \quad x + y = \ln(e^x \times e^y). \quad (1)$$

オイラーの公式

$$e^{i\varphi} = \cos \varphi + i \sin \varphi \quad (2)$$

は、虚数単位 $i = \sqrt{-1}$ を導入した後、三角関数が「exp」および「ln」の別の側面として捉えられることを示している[6]。この視点は、科学者が呼ぶところのexp-log関数、すなわち変数、命名された定数、算術演算、および微分・コンピュータ代数の精神に則ったexpおよびlogから構築される有限式へと自然につながる[31, 32]。古典的な微分代数の設定では、通常、選択された定数体に対して定義され、代数的な付加操作を許容するより広範な意味での初等関数を扱う[7]。すなわち、多項式方程式の根を付加することを意味する（Wolfram言語におけるRoot参照[33]）。このような一般性はここでは必要とされない。本論文では一般的な科学計算機の視点を採用する: まず馴染み深い定数、関数、演算の具体的なリストから出発し、実用的な機能を損なうことなくどこまで簡略化できるかを検討する。正確な開始リストは後述の表1に示されている。

このリスト内での多くの簡略化は当然古典的かつよく知られたものであり、体系的に調査されていないのは、プロセスの最終段階、すなわち単一の二項演算子と単一の区別された定数にまで削減した場合の状態である。特に、いわゆる「壊れた計算機」問題[34]は、利用可能なキーや演算子のセットを制限した計算の一般的な定式化として広く知られている。筆者はこのアプローチを用いて、利用可能な演算子を4つ、3つ、2つ、そして最終的に1つに減らした、次第に「原始的」でありながら完全に機能する計算機の系列を構築した。この演算子はEML (Exp-Minus-Log) であり、

$$\text{eml}(x, y) = \exp(x) - \ln(y). \quad (3)$$

EMLという驚くほど単純な二項演算子を用いることで、任意の標準的な実数の初等関数を、入力変数 $x, y, z, \dots$ と単一の区別された定数1に対して式(3)を反復適用することで表現可能である。この定数は、式(3)に含まれる対数項を $\ln(1) = 0$ によって中和するために必要である。計算は複素数領域で行う必要があり、例えば i や π のような定数を生成するには、 $\ln(-1)$ を評価する必要があるため、内部的には $\mathbb{C}$ を主枝として用いて $\text{eml}(x, y)$ を演算する。

実際のポケット電卓などで外部入力変数が不要な場合、完全な機能を維持するには2つのボタンで十分である: 一つは二項演算子EML、もう一つは終端記号1である。演算子の数をさらに削減することは不可能である。なぜなら、少なくとも一つの二項演算子と一つの終端記号が必ず必要となるからである。このような二項演算子自体が初等関数であるという存在は、やや意外である。

実際、EMLシェフアー演算子(3)はその見かけ通り簡潔であり、原理的にはここで記事を終えてもよい。その帰結はいずれ自然に明らかになるだろう。しかしながら、この演算子の探索に用いた手法(第2節)の説明、中間結果と最終検索結果の提示(第3節)、そして既に視野に見えている潜在的な応用可能性(第4節)について述べることは意義があると考えられる。記事の最後には、簡潔なまとめ、今後の研究方向の可能性、および未解決の問題を記す。

表1: 削減処理を開始する際に用いる定数/変数、単項関数、および二項演算子の初期リスト。この表はまた、論文全体で使用する「基本関数」の具体的な科学的計算機における定義を確定するものである：これらの命名された記号から構成される有限式。 $\exp p(x) = e^x$ は自然対数、 $\ln x$ は逆数、マイナス $(x) = -x$ は符号反転を表す。平方は $\text{sqr}(x) = x^2$ で、 $\sigma(x) = 1/(1 + e^{-x})$ はロジスティックシグモイド関数である。三角関数および双曲線関数とその逆関数は通常意味するところで使用する。基本的な四則演算の二項演算に加え、任意基数の対数 $\log_x y$ 、指数演算 $\text{pow}(x, y) = x^y$ 、算術平均 $\text{avg}(x, y) = (x + y)/2$ 、および斜辺 $\text{hypot}(x, y) = \sqrt{x^2 + y^2}$ を用いる。

種類	要素	個数
定数	$\pi, e, i, -1, 1, 2, x, y$	8
関数	$\exp, \ln, \text{inv}, \text{half}, \text{minus}, \sqrt{}, \text{sqr}, \sigma, \sin, \cos, \tan, \arcsin, \arccos, \arctan, \sinh, \cosh, \tanh, \text{arsinh}, \text{arcosh}, \text{artanh}$	20
演算	$+, -, \times, /, \log, \text{pow}, \text{avg}, \text{hypot}$	8
合計		36

2 手法

私は体系的な「アブレーション」テストを実施し、固定された科学的計算機の初期リストに対して最小限の演算子セットを特定した。このリストは表1に示されており、36個の基本要素から構成される：命名された定数および入力変数、標準的な単項関数、および二項演算である。私はこのリストから1つの要素（定数、関数、または二項演算）を逐次削除し、残ったセットが元の基本要素を再構築できるかどうかを検証した。Mathematica/Wolframのコア言語[33]の命令セット（表2、第2行）は、既に最適化され、約40年にわたって徹底的にテストされてきた最小限のシステムの参照として用いた。研究の発端は、このシステムをさらに削減できるかどうかという純粋な興味からであった。

演算子セットは「完全」と見なされるのは、表1のすべての基本要素を実数軸上で再構築できる場合である。これには三角関数とその逆関数（ $\sin, \cos, \tan, \arcsin, \dots$ ）、双曲線関数とその逆関数（ $\sinh, \cosh, \tanh, \text{arsinh}, \dots$ ）、代数演算（ $\sqrt{}, \text{reciprocal}, \dots$ ）、および基本定数（ $\pi, e, i, -1, 0, 1, 2, \dots$ ）が含まれる。

このようなテストを実行する際の課題は、以下の有理関数生成に関する単純な演習[35]によって示される。 $\text{suc}(x) = x + 1$ （後継演算）、 $\text{pre}(x) = x - 1$ （前任者演算）、および $\text{inv}(x) = 1/x$ （逆数/逆演算）という3つの演算のみを用いて、否定演算、すなわち $-x$ を計算せよという課題である。一見明らかでない解法は以下の通りである：

$$\text{suc}(\text{inv}(\text{pre}(\text{inv}(\text{suc}(\text{inv}(x)))))) = \frac{1}{\frac{1}{\frac{1}{x} + 1} - 1} + 1 = -x.$$

これは我々の探索において遭遇すると予想される数式の典型例である。深さ5～9の入れ子構造を持つ式で、直感的な構築方法が存在しないものである。これらの式を体系的に発見する方法は存在しない

が、現在ではAIを活用した（人間の支援による）総当たり検索手法が用いられている[36]。

直接的な記号的検証は計算量的に非現実的であることが証明された。典型的なコルモゴロフ型の複雑性 K [37] は通常 $K \lesssim 7$ である。実際には $K = 9$ までの範囲で探索が行われた。ここで K は逆ポーランド記法（RPN）計算機プログラムの長さを表し、これは数式と同等である。探索を高速化するため、私は数値ブートストラップ検証と記号的手法を組み合わせたハイブリッド手法を開発した。具体的には、例えば斜辺 $\sqrt{x^2 + y^2}$ のような対象演算が、与えられた演算子セットを用いて表現可能かどうかを確認する。 x や y に対する記号的操作の代わりに、 x と y を代数的に独立な超越定数（例えば $x = \gamma \simeq 0.577216$ （オイラー-マスケローニ定数）、 $y = A \simeq 1.28243$ （グライシャー-キンケリン定数）など）で置換する。その後、 $\sqrt{\gamma^2 + A^2}$ を数値的に評価し、1および式（3）から生成される全ての式と比較する。この方法は、一般的に成立する任意の数式は単一の点においても成立するという単純な観察に基づいている。シャヌエル予想[38]によれば、このような式間の偶然の等式が発生する確率は極めて低く、この手法は数式の等価性候補を選別する信頼性の高いふるいとして機能する。実際には、求める演算子の倍精度数値値を計算した後、その結果を定数認識ソフトウェア（「逆記号計算機」）に入力する。このソフトウェアは、使用手法のバリエーションに応じて、RPN計算機コード形式[39]または式木形式で候補となる数式を返す。探索手順はヒューリスティックなものであり、明らかに誤った数式を排除し、候補となる証人を発見することを目的とする。独立した検証は別途実施され、補足情報（第II部）において、表1のクラスに対する記号的検証、数値的相互検証、および構成的完全性証明の概要が示されている。

詳細に説明すると、探索手順は以下のように機能した。純粋なEML形式で全ての基本演算（表1）を直接探索する（これは数値的手法を用いても計算量的に実行不可能である）代わりに、反復的なブートストラップ手法を採用した：

1. まず2つのリストを用意する：

・検証対象の頑健性を確認するための定数、関数、演算子の集合、例えば $S_0 = \{1, \text{em}\}$

・計算対象の定数、関数、演算子の集合、例えば $C_0 = \{\pi, e, i, -1, \dots, \exp, \ln, \dots, +\}$ （表1参照）。リスト C_0 は、また、私が「全ての基本関数を計算する」と表現する際の定義も規定している。

2. リスト C_i から要素を計算する式を、リスト S_i に含まれる基本演算子のみを用いて探索する

3. 該当する式が見つかった場合、リスト C_i からリスト S_{i+1} へ移動させる。EMLの例では、最初に見つかった結果は $e = \text{eml}(1, 1)$ の計算式である。最初のステップ後、リストは $S_1 = S_0 \cup \{e\} = \{1, \text{eml}, e\}$ と $C_1 = C_0 \setminus \{e\} = \{\pi, i, -1, \dots, \exp, \ln, \dots, +, -, \times, \dots\}$ のようになる

4. 初期リスト C_0 に含まれる全ての基本演算子が再構築されるまで、すなわち C_i が空になるまで、ステップ2-3を繰り返す。具体的には $C_i = \emptyset$ のようになるまで続ける：

検証手順であるVerifyBaseSetは、筆者が作成したMathematica用SymbolicRegressionパッケージ[39]を用いて実装した。処理速度が3桁向上したバージョンも

最近、GPT Codex 5.3によってRust言語に翻訳されており[39]、これによりEMLの再検証が数時間ではなく数秒で可能となった。

アブレーションプロセス、すなわち表1の一部要素を削除した状態で上記の探索を実行すると、次第に小型化された計算機構成が得られる（表2ではCalc 3、2、1、0と命名）。これらの構成はそれぞれ異なる基本演算子や定数を必要とする。一部の構成では任意の入力から必要な定数を独自に生成できる（例えばCalc 2では $x - x = 0$, $e \cdot e^0 = 1 = -1$ などの演算を使用）が、他の構成では特定の定数が必須となる場合がある（例：Calc 1では e または 7 が必要、Wolframでは i が必要、表2の脚注参照）。その後、探索は行き詰まり、連続Sheffer演算子が存在する場合でも、既知の標準的な関数名には含まれていないことが明らかになった。最終的な簡略化においては、多くの実験を通じて培った直感を頼りに、基本的な二項演算子を候補となる単一演算子として列挙し、同様に生成された定数と組み合わせで検証した。各候補ペア{定数、演算子}については、VerifyBaseSet手順を用いてテストを実施した。数多くの失敗と、いくつかの誤検出を除外した後、式(3)で与えられる $\text{eml}(x, y)$ と定数1の組み合わせが、基本関数を再構築するための完全な基底を形成することが明らかになった。後に、EMLが唯一無二の存在ではないことに気づき、その近縁種であるEDL（定数 e を必要とする）や、終端記号として ∞ を使用する第三のバリエーションが存在することが判明した。詳細については次節で述べる。

3 結果

表2に完全な簡略化の過程を時系列順に示す。各構成は表1に列挙された基本演算子から構築されるあらゆる式を計算可能な完全な「科学計算機」に相当する。Calc 3（表2の3行目）は、否定演算と逆数演算を $\exp$ 、 $\ln$ 、加算とともに保持することで、Wolfram言語の基本演算子セット（表2の2行目）を凌駕する最初の計算機構成となった。この6つの基本演算子システムは、 $x + (-x) = 0$ などの演算を通じて独自に定数を生成する能力を備えている。Calc 2は、Calc 3と同様の機能を $\exp$ 、 $\ln$ 演算子と減算のみを用いて実現しつつ、依然として独自に定数を生成する能力を保持している。非可換演算子（減算）が重要であることが判明したのは、この演算子が式の木構造の成長と反転の両方を可能にするためである。

Calc 1は根本的に異なるトップダウンアプローチを採用しており、加算や減算といった低次演算子ではなく、二項指数演算（ランク3の超演算）とその逆演算である二項対数を基本演算子として使用している。この構成では、 e または π を終端定数として用いることができる。広範な調査を行ったものの、Calc 1が動作する他の定数は発見されなかった。

Calc 0は定数 e を $\exp$ 関数自体に吸収させることで、システムを3つの基本演算子にまで削減している。この構成は、単一の二項演算子が存在し得るという強い示唆を与え、さらなる探索を促す結果となった。

最終的なEMLへの還元は、すべての最小構成が逆関数のペア（自己逆関数を含む）と、非可換演算を含むパターンを持っていることを認識することで達成された。入力側で逆関数の組み合わせを試し、出力側で非対称演算を適用した結果、最初の連続的な十分演算子を発見した。1ヶ月後、私は

次のことを認識した：表2: 表1に示した36ボタン式科学電卓からEMLシェファード演算子への段階的削減過程。Calc 3、2、および0は、任意の実数定数または変数を入力として受け入れ、必要な値をすべて内部で生成する。「カウントダウン」列には、入力変数1つを含むすべての異なる基本演算子が計上されている。「定数」欄に「なし」と記載されているシステムでは、任意の実数値を入力として開始できる。参考としてWolfram Mathematicaの基本演算子も記載している。最後の行は、未発見と推測される基本演算子のための仮置きである。最終行の前の行は二項演算子用であり、EMLとは異なり、任意の入力から定数を生成すると仮定されている。最後の行は単項演算子用であり、これはニューラルネットワークにおける十分に検証された活性化関数を持つ優れた特性をすべて保持しつつ、標準の「行列」演算と組み合わせることで初等関数の正確な評価を可能にするものである。

	定数	単項	二項	カウントダウン
基数36	cf. 表1	cf. 表1	cf. 表1	36
Wolfram*	π, e, i	$\ln$	$+, \times, \wedge$	7
Calc 3	なし	$\exp, \ln, x, 1/x$	$+$	6
Calc 2	なし	$\exp, \ln$	$-$	4
Calc 1	e or π	なし	$x^y, \log_x y$	4
Calc 0	なし	$\exp$	$\log_x y$	3
EML	1	なし	$\text{eml}(x, y)$	3
?	なし	なし	?	2
?	なし	?	$+, -, \times$	5

*虚数単位のみで十分である： $-1 = i \times i$ $1 = -1 \times (-1)$ $0 = -1 + 1$ $\pi = -1 \times i \times \ln(-1)$ $e = (-1)^{(i\pi)^{(-1)}}$

さらに少なくとも2つの関連概念が存在する：EDLと-EML：

$$\text{eml}(x, y) = \exp(x) - \ln(y) \quad \text{with constant } 1, \quad (4a)$$

$$\text{edl}(x, y) = \exp(x) / \ln(y) \quad \text{with constant } e, \quad (4b)$$

$$-\text{eml}(y, x) = \ln(x) - \exp(y) \quad \text{with constant } -\infty. \quad (4c)$$

最初の成功した発見的探索実行（cf. 図1）は、この3-line Mathematicaコードで再現可能である：

```
Import["SymbolicRegression.m"] EML [x_, y_] := Exp [x] - Log[y] VerifyBaseSet [{1}, {}, {EML}]
```

このパッケージは、リポジトリから直接インポートすることもできる[39]。コンピュータの処理速度にもよるが、通常1時間以内に、上記の手順によって表1に示された36の基本演算をすべて再生成できる。例えば自然対数は以下のようになる：

$$\ln(z) = \text{eml}(1, \text{eml}[\text{eml}(1, z), 1]), \quad (5)$$

そして同様の手順が続く。生成されるEML式の深さは1（指数関数、 $e^x = \text{eml}(x, 1)$ ）から8（乗算）までの範囲に及び、最も基本的な数学関数ほど深い構造を必要とする傾向がある（表4参照）。より高速かつ徹底的なテスト（複数の実数超越数、正負両方、任意精度チェック）については、VerifyBaseSet[39]のRust再実装版であるrust_verifyが提供されている。

網羅的探索の目的は、候補となる恒等式を発見することのみに留まった。その検証作業は補足情報の第II部に委ねられており、同部ではWolfram Mathematica[33]を用いた発見チェーン全体の記号的簡約化、4つの実装環境（C言語、NumPy、PyTorch、mpmath）にわたる独立した数値的相互検証、および構成的完全性証明の概要が記載されている。

4 使用法と応用

EML式の均一な木構造は、実装および応用における複数の方向性を示唆している。

4.1 EMLコンパイラ

VerifyBaseSetプロシージャの出力は、任意の基本または複合的な初等式をEMLシェファード式（4a）の形で再構築するために必要なデータを提供する（図1参照）。私はPythonで実装したEMLコンパイラのプロトタイプを提供しており、これは数式を純粋なEML形式に変換するものである。EML式はMathematica上で記号的に評価可能であり、またIEEE754規格に準拠した任意の数値計算言語で数値的に評価することもできる。純粋なEML形式は、EML命令のみを備えたハードウェア（あるいはエミュレートされたマシン）上で実行することも可能である。特に、EMLコードは単一命令スタックマシン上で実行可能であり、これは単一ボタン式RPN電卓に極めて類似したアーキテクチャである。純粋なEML形式は、FGAやアナログ回路においても効率的に実装できる可能性がある。

最も単純な入出力例は $\ln x$ 関数によって示される。EML形式へのコンパイル後には、一般的には最も単純な形式は期待できないものの、(5)が得られると予想される。 $\ln$ の同等なRPNコードは、 $K = 7$ 個の命令からなるシーケンスとして表現される。

$$1, 1, x, \text{eml}, 1, \text{eml}, \text{eml},$$

あるいは $\text{eml} \rightarrow E$ をRPN文字列 $11 \times E1EE$ と表記した場合、式木については図2（上部）を参照されたい。

実装に関するいくつかの注意点を述べておく。EMLでコンパイルされた式は、ゼロ点や領域の端点といったごく少数の孤立点を除き、実数軸上の正負両方の値に対して動作する。特に三角関数に関する内部計算は、複素数領域で実行する必要がある。EMLから得られる自然対数の最も単純な形式である(5)は、(3)から導かれる以下の式と等価である：

$$\ln z = e - \log\left(\frac{e^e}{z}\right),$$

複素対数の主枝を標準的な選択方法で用いると、 $1/z$ 項の影響により、負の実数軸上で $2\pi i$ の不連続が生じる。このため、 $\ln(-1) = i\pi$ に基づく標準的な i に関する公式（例えば $\ln(-1) = i\pi$ を使用するもの）は誤った符号を与えることになる。すべての実数 $x \neq 0$ に対して有効な解決策として、EML自体の枝切りを再定義し、 $\ln z$ （およびそれに基づくすべての式、図1参照）が複素対数の主枝の標準的な実装に従うようにする方法がある。別の選択肢として、EMLコンパイラで採用されている方法では、 i の符号を手動で修正する方法がある。

EMLでコンパイルされた数式は、記号処理MathematicaやIEEE754浮動小数点演算環境（例えばCの<math.h>など）において完全に正常に動作する。これは、一部の数式が内部的に拡張実数系の以下の性質に依存しているためである：

$$\ln 0 = -\infty, \quad e^{-\infty} = 0.$$

これらの性質は、Mathematicaでは記号処理によって、また浮動小数点演算ではinf値や符号付きゼロを用いて適切に実装されている。しかし、EML式は一般的に、純粋なPython/Juliaや数値計算用Mathematica環境などではデフォルトでは正常に動作しない。第一のケースでは、特殊な浮動小数点数がトラップされてエラーが発生するためである。ただし、EMLはNumPy [40]やPyTorch [41]では正常に動作する。Mathematicaでは、自動的に拡張可能な浮動小数点数の範囲が設定されており、オーバーフローが発生する[]。興味深いことに、Lean 4証明支援系[42]では異なるアプローチが採用されている。Leanではすべての関数が全射であることが要求されるため、ゼロにおける複素対数にはデフォルトで「無効値」(Complex.log 0 = 0)が割り当てられ、これがEMLの連鎖式を単純に形式化する際の障害となる。結論として、上記のすべての問題（エッジケース）は、通常の浮動小数点演算や記号計算において遭遇する問題と本質的に異なるものではないことを強調しておく。EMLコンパイラは[39]のEML_toolkit/EmL_compilerサブディレクトリでテスト用に利用可能である。詳細は補足情報の第4節「ソフトウェアと再現性」も参照されたい。

4.2 基本関数の二分木表現とアナログ回路モデル

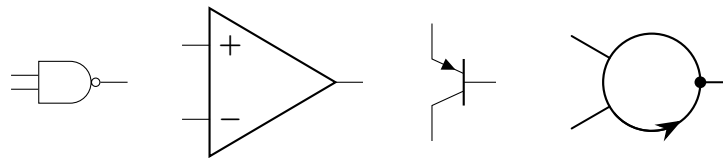
特筆すべきは、EML表記において任意の基本関数の式木は必ず二分木構造を持つことである。文脈自由文法は単純で、 $S \rightarrow 1 \mid \text{eml}(S, S)$ の形式を取る。関数の場合、入力変数は以下のように加算される：

追加の終端記号として扱われる（例えば単変数の場合は x ）。この手法には、標準的な手法に比べて多くの実用的な利点がある。

木構造/回路表現の簡単な具体例を図2に示す。ここで示されているのは自然対数、恒等関数、否定関数 $x \mapsto -x$ 、逆数 $\text{inv}(x) = 1/x$ 、および乗算である。EML木の深さ4で恒等関数を計算できる能力により、一部の入力変数を木構造の下位に移動させることが可能となる（次節参照）。三角関数などの他の基本関数の場合、印刷可能な範囲を超えるほど大きな木構造となるため、表4を参照されたい。

表3: EMLシェッファー演算子は、記号計算およびアナログ計算における新たな基本構成要素としての役割を果たす。ここでドットは出力を表し、式(3)が非可換であるため、矢印はキラル性（方向性）と演算順序を決定する。ドットの直後の最初の入力 は $\exp x$ であり、次に $\ln y$ が続く。ドット付きEMLは「減算」演算を実行する。図2ではこの表記法を混乱を避けるため採用しているが、時計回りのバリエーションも有効である。ここではオペアンプ記号の使用慣習に従っている。

NANDゲート オペアンプ トランジスタ EMLシェッファー演算子



EML演算子を新たな要素として用いた回路（表3）は、アナログ計算[8]において有用である可能性がある。この分野における古典的な課題の一つが、定義済みの多変数基本関数の構築[43]である。EMLコンパイラ（4.1節参照）を使用することで、あらゆる式（図2）をこのような回路に変換でき、そのトポロジーは二分木構造となる。EMLツリーは、一般的な基本関数を統一的に扱うための枠組みを提供する。

4.3 連続最適化による記号回帰

現代の記号回帰（SR）手法[9, 10, 44]は、データから閉形式の表現を発見することを目的としているが、通常は多数の異なる演算子を含む異種文法空間を探索する。任意の基本関数を計算するのに十分な単一の演算子（3）を用いることで、多パラメータの「マスター」式を構築することが可能となる。このマスター式は、二分式木というEML形式の特性により、そのパラメータの特定の組み合わせにおいて、基本関数と等価となる。設計上、このような汎用的なツリーは、与えられた葉ノード数（ツリーの深さ）までのすべての可能な式を含むことになる。これらのツリーは、典型的な数学解析の授業で使用されるものとしては大規模であるが、現代のAI分野で用いられるものと比較すると小規模である。例えば、深さ n の完全二分木は、合計 $2^n - 1$ 本の枝と 2^n 個の葉ノードを持つ。パラメータ数が兆単位[formula id=3]に達する最大級のトランスフォーマーの場合、同等のツリーの深度は40となる。表4の大きな値は、最適化されていないプロトタイプEMLコンパイラ（4.1節サブセクション参照）によるものであるが、直接的な網羅的探索を行うことで、右端の列が示すようにより簡潔な式が得られる。

マスター式は以下のように構築できる。簡略化のため単変数の場合を示すが、この構築手法は任意の数の入力変数に拡張可能である。式の両入力 は

表4: EMLツリー表現における各種関数の複雑度。EMLコンパイラ列には、EMLコンパイラから生成された式のRPNコード長 K を示す。EML式の K 値は、例えばMathematicaのLeafCount関数を用いて計算できる。恒等関数 x の場合、コンパイラは直接 x を返す（葉ノード数1）。最短の非自明なEML式の葉ノード数は9である。最終列には、最短式を求めるための直接的な網羅的探索の結果を示している。括弧内の数値は、拡張実数（浮動小数点表現における $\pm\text{inf}$ ）を使用しない場合の式の長さを示す。探索がタイムアウトした場合、 K の下限值が示される。

		定数	EMLコンパイラ	直接探索	
		1	1	1 (1)	
		0	7	7 (7)	
		-1	17	15 (17)	
		2	27	19 (19)	
		-2	43	27 (27)	
		1/2	91	29 (35)	
		-1/2	107	31 (37)	
		2/3	143	39 (39)	
		-2/3	159	45 (47)	
		$\sqrt{2}$	165	>47	
		i	131	>55	
		e	3	3	
		π	193	>53	
関数		EMLコンパイラ	直接探索		
x		1	9	演算子	
e^x		3	3		
$\ln x$		7	7	-	83
-x		57	15	$x + y$	11 (11)
$\frac{1}{x}$		65	15	$x \times y$	19 (19)
$x - 1$		43	11	x/y	17 (17)
$x + 1$		27	19	x^y	105
$x/2$		131	27	$\log_x y$	49
$2x$		131	19	$(x + y)/2$	25
$\sqrt{x}$		139	43 $\geq ? > 35$	$x^2 + y^2$	29
x^2		75	17		287
					>27

$\text{eml}(x, y)$ は以下のいずれかとなる: 1、入力変数 x 、または前のEML計算結果 f 。このような一般的な入力を線形結合で表現すると以下ようになる:

$$\alpha_i + \beta_i x + \gamma_i f. \quad (6)$$

特定の値を設定することで、3つのケースすべてを復元できる

- ・ $\alpha_i = 1 \ \beta_i = 0 \ \gamma_i = 0$ の場合: 1
- ・ $\alpha_i = 0 \ \beta_i = 1 \ \gamma_i = 0$ の場合: x
- ・ $\alpha_i = 0 \ \beta_i = 0 \ \gamma_i = 1$ の場合: f

最低次のツリーレベル（葉ノード）では1と x のみが存在するため、パラメータ γ_i は存在しない。
。例として、完全なレベル2マスター式を示す:

$$F(x) = \text{eml} \left[\alpha_1 + \beta_1 x + \gamma_1 \text{eml}(\alpha_3 + \beta_3 x, \alpha_4 + \beta_4 x), \right. \\ \left. \alpha_2 + \beta_2 x + \gamma_2 \text{eml}(\alpha_5 + \beta_5 x, \alpha_6 + \beta_6 x) \right].$$

$\alpha_1 = 0 \ \beta_1 = 1 \ \gamma_1 = 0$ および $\alpha_2 = 1$ を1に設定し、 $\beta_2 = \gamma_2 = 0$ とすると、 $\exp(x)$ が得られる。
 $\alpha_1 = \alpha_2 = 1$ とすべての $\beta_i = \gamma_i = 0$ を用いることで定数 e が得られる。 $\alpha_1 = \beta_1 = 0 \ \gamma_1 = 1$ を1に設定し、 $\alpha_2 = 1 \ \beta_2 = \gamma_2 = 0 \ \alpha_3 = 0 \ \beta_3 = 1 \ \gamma_3 = 0$ および $\alpha_4 = 1 \ \beta_4 = \gamma_4 = 0$ とすると、二重指数関数 $\exp(e^x)$ が得られる。レベル2マスター式のパラメータ総数は14である。一般に、レベル n マスター式には $5 \times 2^n - 6$ 個のパラメータが存在する（詳細は補足情報のパートIIIを参照）。

理論的にはより一般的な基本関数(+, -, /, **) (sin, cosなど) を用いてこのようなマスター式を構築することも可能ではあるが、その場合非常に複雑になり、規則的な構造を欠くことになるだろう。さらに、実際のSR（特殊相対性理論）の実践では、通常、操作の縮小されたサブセットを扱うため、選択した操作セットが不完全であるリスク、すなわちすべての基本関数を表現できない可能性がある。EMLマスター式は設計上完全なものである。

式(6)におけるパラメータ数はスイッチ関数を用いることで2つに削減可能である。あるいは、より現代的なアプローチとして、パラメータ $\alpha_i \ \beta_i \ \gamma_i$ をロジットとして扱い、ソフトマックス関数[45]に通して $\alpha_i + \beta_i + \gamma_i = 1$ に正規化された確率値に変換する方法もある。以下の2つの例では、両方の手法を使用する。

最も単純な概念実証は、上記の構造を持つ完全なレベル3バイナリツリーを用いて、基本的な基本関数 $\ln x$ から得られた数値データに適合させることで得られる。シンプレックス再パラメータ化を適用することで、自由パラメータの数は34から20に減少した。私はMathematica[33]の「ブラックボックス」最適化アルゴリズムであるNMinimizeを用いて、すべての重み値を求めることに成功した。平均二乗誤差による適合誤差は数値精度の範囲内であり、重み値をシンプレックスの最も近い頂点に丸めた後の結果式は正確に $\ln x$ となる。これは提供されたデータ範囲内だけでなく、その範囲を超えても同様である。上記の例では、一般化/外挿がほぼ完全に行われている。詳細はEMLツールキット[39]のWolfram Mathematicaノートブック「Log fit.nb」および補足情報（第III.B節）を参照されたい。

実際には、このような単純な一行コードのアプローチではスケールしない。そこで、より最近の機械学習技術を用いた複数の実験を行った。対象関数としては、EMLから構成された二変数関数 $\ln(e - \ln(e^x - \ln y))$ などを使用した。DTYPE = torch.complex128データ型を使用するPythonコードを作成し、EMLネットワークの訓練中に主に生じた問題は、多重に構成された指数関数による範囲オーバーフロー、および複素数演算の特定の実装に起因する浮動小数点誤差 (NaN) であった。これらの問題は、 $\exp(x)$ の引数と値に対してクランプ処理を施し、実部と虚部の両方を注意深く (torchの自動微分機能を損なわない範囲で) 検査することで解消できる。式(6)のパラメータはロジットとして扱った。最適化は多段階プロセスで実施した：まず確率的勾配最適化アルゴリズム (Adam) を用いた通常の訓練を行い、次に式(6)の重みを0または1に近づける強化フェーズを実施した。最後に、重み値を厳密な記号値にクランプした。系統的な実験 (1000回以上の実行で、シード値と初期化戦略を変化させた) の結果、ランダムな初期値からの盲目的回復は、深さ2では100%、深さ3-4では約25%、深さ5では1%未満の成功率であった。深さ6では、448回の試行で盲目的回復は確認されなかった。成功した場合、丸めた重み値から得られる平均二乗誤差は機械イプシロンの二乗 ($\sim 10^{-32}$) と一致し、これは厳密な記号値による回復と整合する。注目すべきは、正しいEMLツリーの重み値にガウスノイズを加えた場合でも、最適化は深さ5および6のツリーにおいても100%の試行で正確な値に収束することである。これは、EMLツリーの表現が有効であり、正しい引力領域が存在することを示している。ランダムな初期値からこれらの領域を発見することは、深さが増すにつれて困難になる。もしEMLツリーの収束性を概念実証の域を超えて改善できる方法、例えば式(4)と同様の別の二項演算子でより優れた特性 (非指数関数的な漸近挙動、領域問題の解消など) を持つものを見つけれれば、勾配ベースの手法[46]を用いてデータの記号回帰を実現することが可能となる。系統的な訓練実験、コード、および詳細については補足情報を参照されたい。

5 結論と未解決課題

演算子EML (式3) は、実数の基本関数を構築・評価するための単一の十分な基本要素を提供する。したがって、このような関数から構成される広範なクラスの計算もEML形式で表現可能である。これは唯一無二のものではなく、EDL (式4b) や引数を入れ替えた形式 $\text{eml}(y, x)$ (式4c) など、類似の特性を示す近似的な変種が複数存在する。このような演算子はさらに存在する可能性がある。おそらく、より優れた特性 (式3よりも便利な性質) を持つ連続的な演算子のファミリー全体が発見されるのを待っているのかもしれない。例えば、定数の一つ $-\infty, 1, e, \dots$ が常に終端記号として存在するという要件は、その使用を標準ニューラルネットワークやNANDゲートと比較してより洗練されていないものとし、複雑さを増す (4.3節参照)。後者は「あらゆる入力」から0と1を生成する有用な特性を持つ。EML演算子にはこの有用な特性がない。EML型の二項シェッファー演算子で、区別された定数とペアを組むことなく動作するものが存在するかどうかは、未解決の問題である。このような可能性がいかなる候補に対しても不可能であることを証明することは容易ではない： $f(x, x)$ が定数であることで十分だと予想されるかもしれないが、 $B(x, y) = x - \frac{y}{2}$ を考えると、 $B(x, x) = \frac{x}{2}$ は成立するが $B(B(x, x), x) = 0$ が成立するようなケースが存在する。このような罠が存在することが、本研究において系統的な探索が不可欠である理由を示している。

³E.g. $1 \equiv \text{NAND}(x, \text{NAND}(x, x))$ 実際には、この特性を使用することはほとんどない。通常は、デジタル回路におけるGNDや V_{cc} など、直接利用可能な0と1を使用する。

三項演算子 $T(x, y, z) = e^x / \ln x \times \ln z / e^y$ において、 $T(x, x, x) = 1$ が次の分析対象候補として挙げられる[47]。

[47]単変数シェフアー演算子で、ニューラルネットワークの活性化関数[48]としての役割と、すべての基本関数を生成する機能を同時に果たすものが存在し得るかどうかは、未解決の問題である（補足資料5節参照）。

基本関数のEML表現では、実数数学の計算に複素数演算が必要となる点を指摘する意見もあるだろう。量子コンピューティングが複素振幅を用いて実確率を計算するのと同様に、EMLでは複素中間値を用いて実数の基本関数を計算する。このことは避けられないように思われる。我々は何らかの方法で、虚数単位 i および $\sqrt{-1}$ 、さらにオイラーの公式（2）を用いてすべての三角関数/双曲線関数を計算しなければならない。この目的のために、 $\ln x$ を $x < 0$ に対して使用する。実数領域のみで動作する連続的なシェフアー演算子を実現することは不可能であるようだ。 $\exp / \ln$ の代わりに三角関数/双曲線関数とその逆関数のペアを使用するなど、代替手段を模索したが何も見つからなかった。驚くべきことに、内部的に複素数を使用する必要があるにもかかわらず、この要件はコンピュータ代数システムや数値シミュレーションにおいて式（3）を実際に使用する場合の問題をごくわずかなものに留めている。

標準的な活性化関数自体が基本関数であるため、従来のニューラルネットワークはすべてEMLツリーアーキテクチャの特殊なケースと言える。現在のネットワークは記号代数[49]や桁レベルの算術[50]を学習できるが、その内部メカニズムは依然として不透明である[11]。また、連続的な実数値演算としての基本関数の効率的な厳密評価は、依然として彼らの能力の範囲外にある。EML表現はさらに進んだ機能を提供する：第4.3節で示すように、学習済みの重みは正確な2値にスナップすることができ、近似値とともに閉形式の基本部分式を復元する。この手法が成功した場合、発見された回路は基本関数の表現として解釈可能となる——これは従来のアーキテクチャでは得られない解釈可能性の形態である。

AI利用に関する開示事項

本論文の核心的なアイデアであるEMLシェフアー演算子の発見、検証方法論、および結果はすべて著者独自の研究成果である。大規模言語モデル（最近のClaude、Grok、Gemini、ChatGPTを含む）は主に言語編集およびコーディング支援のために使用された。

データの利用可能性

図表および数値結果を生成するために使用したコード、スクリプト、および補足的な再現性資料は、SymbolicRegressionPackageリポジトリ[39]で公開されている。提出版の正確なアーカイブコピーはZenodo DOI: 10.5281/zenodo.19183008 (<https://doi.org/10.5281/zenodo.19183008>) に保存されている。このアーカイブパッケージには、報告された結果を再現するために必要なソースコード、スクリプト、およびREADMEファイルが含まれている。

謝辞

計算リソースの一部はGoogle Cloud Research Creditsによって提供された。

補足情報

詳細な3部構成の補足情報は、別個のSI Appendix PDFとして提供されている。

参考文献

[1] William H. Press, Saul A. Teukolsky, William T. Vetterling, Brian P. Flannery. 『Numerical Recipes: 科学計算の Art』。Cambridge University Press、ケンブリッジ、第3版、2007年。ISBN 978-0-521-88068-8。

[2] Henry Maurice Sheffer. ブール代数の5つの独立公理体系と論理定数への応用。『American Mathematical Society Transactions』、14巻4号、481-488頁、1913年。doi: 10.1090/S0002-9947-1913-1500960-1。

[3] John Napier. 『Mirifici Logarithmorum Canonis Descriptio』。Andrew Hart、エディンバラ、1614年。

(日本語訳: 『驚異の対数表の記述』) [4] Henry Briggs. 『Arithmetica Logarithmica』。ロンドン、1624年。

(日本語訳: 『対数算術』) [5] Roger Cotes. 『Harmonia Mensurarum, sive Analysis & Synthesis per Rationum & Angulorum Mensuras Promotae: Accedunt Alia Opuscula Mathematica』。ケンブリッジ、1722年。(日本語訳: 『尺度の調和: 数と角度による測定に基づく解析と総合、およびその他の数学論文集』) ※著者名の日本語表記は標準的な学術表記に従った。

[6] Joseph Liouville. 「超越関数の一クラスの積分に関する研究」。『Journal für die reine und angewandte Mathematik』(クレレ誌)、第13巻、93-118頁、1835年。doi: 10.1515/crll.1835.13.93。

(日本語訳: 「純粋および応用数学雑誌」) [7] Joseph Fels Ritt. 『有限項による積分: リウヴィルの初等的手法理論』。コロンビア大学出版局、ニューヨーク、1948年。

(日本語訳: 「有限項による積分: リウヴィルの初等的手法理論」) [8] Bernd Ulmann. 『アナログ計算』。De Gruyter Oldenbourg、ベルリン・ボストン、2022年。ISBN 9783110787740。doi: doi:10.1515/9783110787740。URL <https://doi.org/10.1515/9783110787740>。

(日本語訳: 「有限項による積分: リウヴィルの初等的手法理論」) [9] Silviu-Marian Udrescu and Max Tegmark. 『AI Feynman: 物理原理に基づく記号回帰手法』。『Science Advances』、6巻16号: eaay2631、2020年。doi: 10.1126/sciadv.aay2631。(日本語訳: 「AI Feynman: 物理原理に基づく記号回帰手法」)

[10] Miles Cranmer, Alvaro Sanchez-Gonzalez, Peter Battaglia, Rui Xu, Kyle Cranmer, David Spergel, and Shirley Ho. 『深層学習における帰納的バイアスを用いた記号モデルの発見』。『Advances in Neural Information Processing Systems』、33巻: 17429-17442、2020年。

(日本語訳: 「深層学習における帰納的バイアスを用いた記号モデルの発見」) [11] Ziming Liu, Yixuan Wang, Sachin Vaidya, Fabian Ruehle, James Halverson, Marin Soljačić, Thomas Y. Hou, and Max Tegmark. 『KAN: コルモゴロフ-アーノルドネットワーク』。『International Conference on Learning Representations (ICLR)』、2025年。URL: <https://openreview.net/forum?id=Ozo7qJ5vZi>。(日本語訳: 「KAN: コルモゴロフ-アーノルドネットワーク」)

[12] Jan Łukasiewicz. 『数学的論理の基礎』。Pergamon Press, Oxford, 1963年。原著『Elementy logiki matematycznej』(1929年)の英語訳。

(日本語訳: 『数学的論理の基礎』) [13] H. S. Black. 『安定化フィードバック増幅器』。『The Bell System Technical Journal』、13巻1号: 1-18頁、1934年。doi: 10.1002/j.1538-7305.1934.tb00652.x。(日本語訳: 「安定化フィードバック増幅器」)

- [14] Vinod Nair and Geoffrey E. Hinton. 『Rectified linear unitsは制限付きボルツマンマシンを改善する』。『Proceedings of the 27th International Conference on Machine Learning (ICML 2010)』, pp. 807-814, 2010年。(日本語訳: 「Rectified linear unitsは制限付きボルツマンマシンを改善する」)
- [15] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. 『Deep Learning』。MIT Press, マサチューセッツ州ケンブリッジ, 2016年。ISBN 9780262035613. URL <https://www.deeplearningbook.org/>。(日本語訳: 『深層学習』)
- [16] Stephen Wolfram. 『A New Kind of Science』。Wolfram Media, 2002年。ISBN 1-57955-008-8. URL <https://www.wolframscience.com/nks/>。(日本語訳: 『新しい種類の科学』)
- [17] M. Schönfinkel. 『数学的論理の構成要素について』。『Mathematische Annalen』, 92巻3号: 305-316頁, 1924年。ISSN 1432-1807. doi: 10.1007/BF01448013. URL <https://doi.org/10.1007/BF01448013>。(日本語訳: 「数学的論理の構成要素について」)
- [18] H.B. Curry and R. Feys. 『Combinatory Logic』。『Combinatory Logic』シリーズ 第1巻。North-Holland Publishing Company, 1958年。URL <https://books.google.pl/books?id=fEnuAAAAMAAJ>。(日本語訳: 『組合せ論理』)
- [19] Yves Lafont. 『相互作用コンビネータ』。『Inf. Comput.』, 137巻1号: 69-101頁, 1997年8月。ISSN 0890-5401. doi: 10.1006/inco.1997.2643. URL <https://doi.org/10.1006/inco.1997.2643>。(日本語訳: 「相互作用コンビネータ」)
- [20] Michał Baczynski, Pedro Berruezo, Piotr Helbin, Sebastia Massanet, Wanda Niemyska, and Daniel Ruiz-Aguilera. 『ファジィ論理におけるシェファーストローク演算について』。『Fuzzy Sets and Systems』, 431巻: 110-128頁, 2022年。ISSN 0165-0114. doi: <https://doi.org/10.1016/j.fss.2021.05.003>. URL <https://www.sciencedirect.com/science/article/pii/S0165011421001834>。(日本語訳: 「ファジィ集合とシステム」)
- [21] Oleg Mazonka and Alex Kolodin. 『subleqに基づくシンプルなマルチプロセッサコンピュータ』, 2011年。URL <https://arxiv.org/abs/1106.2593>。(日本語訳: 「arXiv論文」)
- [22] J. H. Conway. 『FRACTRAN: 算術演算のためのシンプルな汎用プログラミング言語』, pp. 4-26. シュプリンガー・ニューヨーク社, ニューヨーク, NY, 1987年。ISBN 978-1-4612-4808-8. doi: 10.1007/978-1-4612-4808-8.2. URL https://doi.org/10.1007/978-1-4612-4808-8_2。(日本語訳: 「Springer出版論文」)
- [23] Matthew Cook. 『初歩的なセルオートマトンにおける普遍性』。Complex Systems, 15(1): 1-40, 2004年。doi: 10.25088/ComplexSystems.15.1.1。(日本語訳: 「Complex Systems誌論文」)
- [24] B. Linsky. 『『プリンキピア・マテマティカ』の発展: バートランド・ラッセルによる第2版用原稿とノート』。ケンブリッジ大学出版局, 2011年。ISBN 9781139497336. URL <https://books.google.pl/books?id=EHCMuK4gS-MC>。(日本語訳: 「Google Books掲載書籍」)
- [25] Francis H. C. Crick and James D. Watson. 『デオキシリボ核酸の構造が持つ遺伝学的意義』。Nature, 171(4361):964-967, 1953年。doi: 10.1038/171964b0。(日本語訳: 「Nature誌論文」)

[26] D. Smith, J. S. Myers, C. S. Kaplan, and C. Goodman-Strauss. 『非周期的単一タイル』 . Combinatorial Theory, 4(1), 2024年. doi: 10.5070/C64163843. URL <https://escholarship.org/uc/item/3317z9z9>. (日本語訳: 「Scholarship.org掲載論文」)

[27] Andrzej Odrzywolek. 『完璧なイグルーの作り方』 . Eureka, 63:1001110-1001111, 2014年. (日本語訳: 「Eureka誌論文」)

[28] E. Fehlberg. 『ステップ制御を備えた4次およびそれ以下の古典的ルンゲ＝クッタ公式と熱伝導問題への応用』 . Computing, 6(1):61-71, 1970年. ISSN 1436-5057. doi: 10.1007/BF02241732. URL <https://doi.org/10.1007/BF02241732>. (日本語訳: 「DOI掲載論文」)

[29] 高橋秀俊・森正武. 『数値積分のための二重指数関数公式』 . 数理科学研究所紀要, 9(3):721-741, 1974年. doi: 10.2977/prims/1195192451. (日本語訳: 「PRIMS掲載論文」)

[30] James W. Cooley and John W. Tukey. 『複素フーリエ級数の機械計算アルゴリズム』 . Mathematics of Computation, 19(90):297-301, 1965年. doi: 10.1090/S0025-5718-1965-0178586-1. (日本語訳: 「Mathematics of Computation掲載論文」)

[31] John Shackell. 『exp-log関数の成長評価』 . Journal of Symbolic Computation, 10(6):611-632, 1990年. ISSN 0747-7171. doi: [https://doi.org/10.1016/S0747-7171\(08\)80161-7](https://doi.org/10.1016/S0747-7171(08)80161-7). URL <https://www.sciencedirect.com/science/article/pii/S0747717108801617>. (日本語訳: 「ScienceDirect掲載論文」)

[32] Daniel Richardson, Bruno Salvy, John Shackell, and Joris van der Hoeven. 『exp-log関数の漸近展開』 . 技術報告書 RR-2859, INRIA, 1996年. URL <https://inria.hal.science/inria-00073832.inria-00073832>. (日本語訳: 「INRIA技術報告書」)

[33] Wolfram Research, Inc. 『Mathematica』 , バージョン14.3, 2025年. URL <https://www.wolfram.com>. Mathematica. Champaign, IL, 2025年. (日本語訳: 「Wolfram Research社提供のMathematicaソフトウェア」)

[34] Dan Finkel. 『壊れた電卓を使ったウォームアップ演習』 . <https://mathforlove.com/lesson/broken-calculator-warmup/>, 2017年. 2025年11月18日アクセス.

(日本語訳: 「数学教育サイト『mathforlove.com』掲載の教材」) [35] Donald J. Newman. 『問題セミナー』 . 『数学・物理学史資料集』 . シュプリンガー・フェアラーク社, ニューヨーク, 1982年. ISBN 0-387-90765-3. (日本語訳: 「数学史・物理学史分野の学術書」)

[36] Bartosz Naskrecki and Ken Ono. 『人工知能時代における数学的発見』 . Nature Physics, 21巻10号: 1504-1506頁, 2025年10月. DOI: 10.1038/s41567-025-03042-0. URL: <https://doi.org/10.1038/s41567-025-03042-0>. (日本語訳: 「Nature Physics誌掲載の学術論文」)

[37] A. N. コルモゴロフ. 『情報の定量的定義に関する三つのアプローチ』 . 『情報伝達問題』 , 1巻1号: 1-7頁, 1965年. (日本語訳: 「ソ連の数学者による情報理論に関する学術論文」)

[38] Giuseppina Terzo. 『指数環におけるチャンネル予想の帰結について』 . 『代数通信』 , 36巻3号: 1171-1189頁, 2008年. DOI: 10.1080/00927870701410694. URL: <https://doi.org/10.1080/00927870701410694>. (日本語訳: 「イタリアの数学者による代数理論に関する学術論文」)

[39] Andrzej Odrzywolek. SymbolicRegressionPackage: 記号回帰のための基本構成要素. <https://github.com/VA00/SymbolicRegressionPackage>, 2026年. EMLツールキットを含む。(日本語訳:「ポーランドの数学者が開発した記号回帰用のオープンソースパッケージ」)

[40] Charles R. Harris, K. Jarrod Millman, Stefan J. van der Walt, Ralf Gommers, Pauli Virtanen, David Cournapeau, Eric Wieser, Julian Taylor, Sebastian Berg, Nathaniel J. Smith, Robert Kern, Matti Picus, Stephan Hoyer, Marten H. van Kerkwijk, Matthew Brett, Allan Haldane, Jaime Fernández de l Río, Mark Wiebe, Pearu Peterson, Pierre Gérard-Marchant, Kevin Sheppard, Tyler Reddy, Warren Weckesser, Hameer Abbasi, Christoph Gohlke, および Travis E. Oliphant. 『NumPyによる配列プログラミング』. 『Nature』, 585巻7825号: 357-362頁, 2020年9月. DOI: 10.1038/s41586-020-2649-2. (日本語訳:「Pythonの数値計算ライブラリNumPyに関する学術論文」)

[41] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Köpf, Edward Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, および Soumith Chintala. 『PyTorch: 命令型スタイルの高性能深層学習ライブラリ』. 『Advances in Neural Information Processing Systems 32』, pp. 8024-8035, 2019年. (日本語訳:「深層学習向けPythonライブラリPyTorchに関する技術論文」)

[42] Leonardo de Moura および Sebastian Ullrich. 『lean 4定理証明器およびプログラミング言語』. 『Automated Deduction - CADE 28: 2021年第28回自動推論国際会議』 (オンライン開催、2021年7月12-15日) 論文集, pp. 625-635, ベルリン・ハイデルベルク, 2021年. Springer-Verlag. doi: 10.1007/978-3-030-79876-5_37. URL https://doi.org/10.1007/978-3-030-79876-5_37. (日本語訳:「自動推論システムにおける定理証明器lean 4とその関連プログラミング言語に関する研究論文」)

[43] Bernd Ulmann. 『0と1を超えて - 21世紀におけるアナログ計算』. International Journal of Parallel, Emergent and Distributed Systems, 39(2):139-151, 2024. doi: 10.1080/17445760.2023.2296672. URL <https://doi.org/10.1080/17445760.2023.2296672>. (日本語訳:「デジタル計算の限界を超えるアナログ計算技術の可能性に関する最新研究」)

[44] Miles Cranmer. 『PySRおよびSymbolicRegression.jlを用いた科学分野向け解釈可能な機械学習』, 2023. (日本語訳:「PySRライブラリとSymbolicRegression.jlフレームワークを活用した、科学的応用に適した解釈可能な機械学習手法に関する研究」)

[45] Eric Jang, Shixiang Gu, および Ben Poole. 『Gumbel-Softmaxを用いたカテゴリカル再パラメータ化』. 第5回国際学習表現会議 (ICLR) 論文集, 2017. URL <https://openreview.net/forum?id=rkE3y85ee>. (日本語訳:「カテゴリカル変数を扱う機械学習モデルにおけるGumbel-Softmax法を用いた効果的なパラメータ化手法に関する研究」)

[46] Brenden K. Petersen, Mikel Landajuela, T. Nathan Mundhenk, Cláudio P. Santiago, Sookyung Kim, および Joanne T. Kim. 『深層記号回帰: リスク志向型ポリシー勾配によるデータからの数学的表現の復元』. 国際学習表現会議 (ICLR), 2021. URL <https://openreview.net/forum?id=m5Qsh0kBQG>. (日本語訳:「深層学習と記号回帰技術を統合した、データから数学的表現を自動生成する革新的な手法に関する研究」)

[47] Andrzej Odrzywolek. 『初等関数に対する三値シェファード演算子の可能性』. Acta Physica Polonica B, 2026. 準備中.

（日本語訳：「初等関数の解析に三値演算子を適用した新たな数学的枠組みに関する研究」）[48] Shiv Ram Dube y, Satish Kumar Singh, および Bidyut Baran Chaudhuri. 『深層学習における活性化関数：包括的な調査とベンチマーク』 . Neurocomputing, 503: 92-108, 2022. doi: 10.1016/j.neucom.2022.06.111. （日本語訳：「深層学習モデルにおける活性化関数の理論的基礎と実用的評価に関する体系的研究」）

[49] Guillaume Lample and François Charton. 『記号数学のための深層学習手法』 . Proceedings of the 8th International Conference on Learning Representations (ICLR), 2020. URL <https://openreview.net/forum?id=S1eZYeHFDS>. （日本語訳：「深層学習技術を記号数学問題に適用するための新しいアプローチに関する研究」）

[50] Nayoung Lee, Kartik Sreenivasan, Jason D. Lee, Kangwook Lee, および Dimitris Papailiopoulos. 『小規模トランスフォーマーモデルへの算術演算の学習』 . The Twelfth International Conference on Learning Representations (ICLR), 2024. URL <https://openreview.net/forum?id=dsUB4bst9S>. （日本語訳：「小規模ニューラルネットワークアーキテクチャにおける基本的な算術演算の効果的な学習手法に関する研究」）

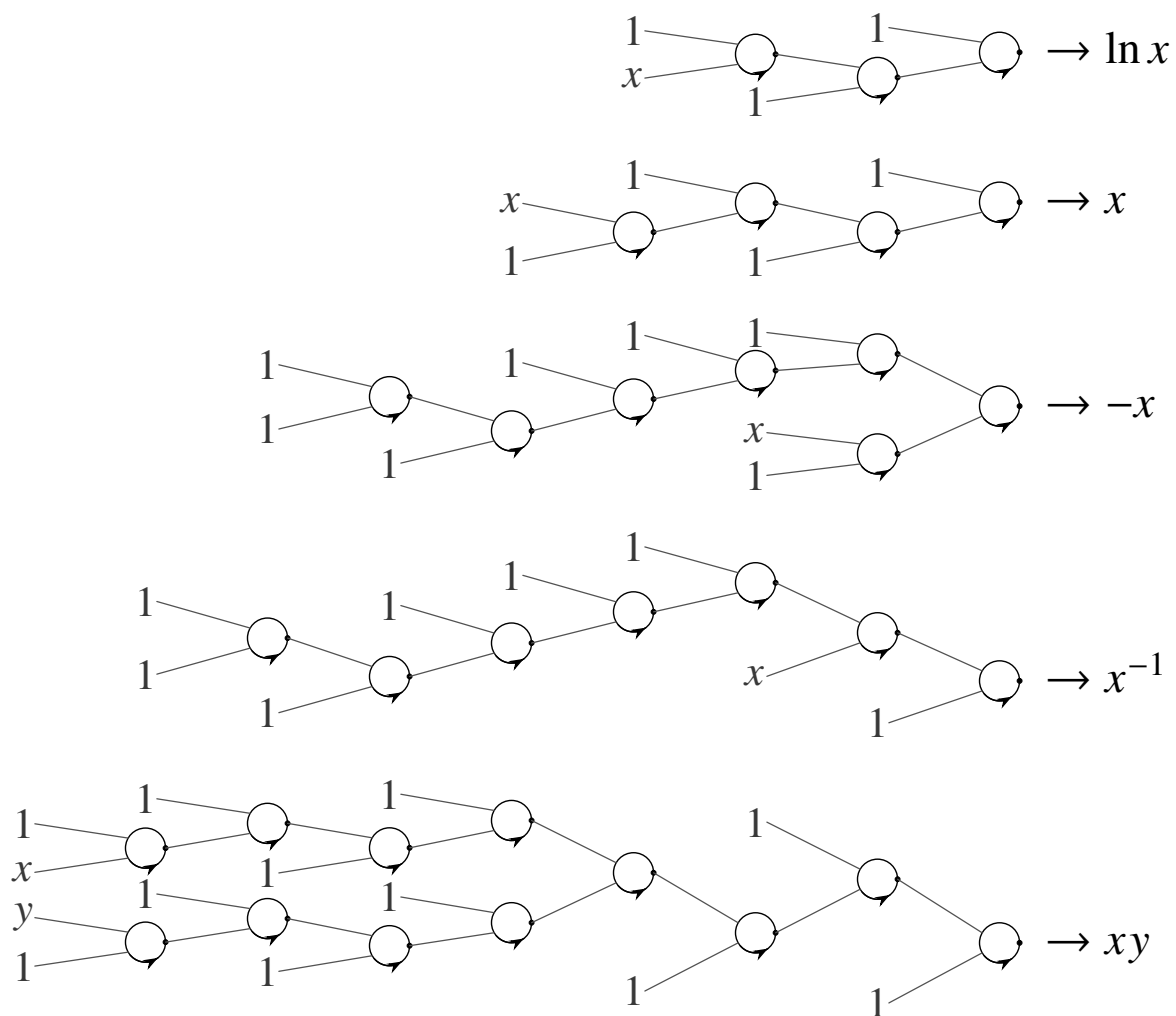


図2: いくつかの重要な最も単純な数式に相当する二値EMLツリーの具体例。